

Poly Agent 垂类 Torch 小模型导入与使用操作手册

适用对象：需要把自研 PyTorch / TorchScript / state_dict 小模型接入 Poly Agent 垂类预测模型工作台的算法开发者、模型交付人员和平台运维人员。

适用入口：任务提交 -> 垂类预测模型，页面路由 /vertical-prediction。

适用日期：2026-07-24，基于当前 Poly Agent 本地代码库实现。

1. 当前结论

Torch 小模型可以接入。

当前平台的上传算法运行时是 Python 3.11 的本地 sandbox 子进程。平台接收标准算法 ZIP，校验 polyagent.algorithm.yaml、样例输入、入口函数和输出结构，通过后完成构建、部署、激活。激活后的模型可以在垂类预测模型工作台、人工 Workflow 和 AutoResearch 中调用。

接入 Torch 模型时需要注意这些边界：

项目	当前能力 / 约束
运行语言	Python 3.11
运行方式	local_sandbox_runtime，短生命周期 Python 子进程
默认硬件	CPU；契约里默认 gpu: false
Torch 依赖	运行环境里必须已预装 torch；当前构建流程记录 requirements 摘要，不自动创建隔离环境安装依赖
小权重包	ZIP 允许 .pt、.bin 等后缀；整个 ZIP 最大 20MB
.pth 权重	当前 ZIP 白名单未包含 .pth；建议导出为 .pt，或把 .pth 放到受管资源路径
大权重	不建议放进 ZIP；通过“资源管理”登记服务器/挂载路径，再在算法契约中声明 resource_assets
网络/密钥	sandbox 环境变量白名单较窄，不应依赖用户本机密钥或在线下载权重
入口函数	load(context) 可选加载模型；predict(inputs, context, model) 执行预测并返回 dict

2. 接入角色分工

角色	负责事项
算法开发者	提供模型权重、推理代码、输入输出说明、样例输入、依赖列表和来源信息
平台接入人员	检查包结构、契约、依赖、资源路径和样例 dry-run

角色	负责事项
平台运维人员	在后端 Python 环境预装 torch 等依赖，登记大模型权重目录，维护资源根目录白名单
使用者	在垂类预测模型页面选择已激活版本，填写输入或上传文件并查看结果

3. 接入前准备清单

交付前请准备以下材料：

材料	必需	说明
src/handler.py	是	包含 load 和 predict
tests/sample_input.json	是	样例输入必须能完成 dry-run
requirements.txt	建议	只写 PyPI 依赖名和版本，不写 git/http/file 来源
model/model.pt	小模型可选	小于包大小限制时可随 ZIP 打包
受管资源目录	大模型可选	放置 .pt、.pth、tokenizer、归一化参数等
polyagent.algorithm.yaml	是	标准 ZIP 必须包含
README.md	建议	说明模型用途、输入输出、限制和引用
开发者/机构/引用信息	建议	用于页面来源标注和结果追溯

推荐模型交付格式：

- 1. 优先交付 state_dict，不要交付完整 Python 对象 pickle。
- 2. 权重文件优先使用 .pt。
- 3. 模型结构定义放在 src/handler.py 或 src/model.py 中。
- 4. 推理固定使用 CPU，加载时使用 map_location="cpu"。
- 5. 把特征工程、标准化参数、类别映射一并交付，避免平台侧猜测。

4. 标准 ZIP 目录结构

小模型随包交付时推荐结构：

```
polyagent.algorithm.yaml
requirements.txt
README.md
src/
  handler.py
model/
  model.pt
tests/
  sample_input.json
```

大模型用受管资源时推荐结构：

```
polyagent.algorithm.yaml
requirements.txt
README.md
src/
  handler.py
tests/
  sample_input.json
```

大权重放在平台后端可访问路径，例如：

```
.runtime/algorithm-resources/polymer_torch_tg/
  model.pt
  feature_stats.json
  label_mapping.json
```

如果需要使用 .pth 文件，不要放入 ZIP；放入受管资源目录，并在资源管理页面登记。

5. Torch 推理入口模板

以下示例适用于结构-性质预测类小模型。实际接入时替换 TorchRegressor、featurize 和输出字段即可。

```

from __future__ import annotations

import json
from pathlib import Path

import torch
from torch import nn

class TorchRegressor(nn.Module):
    def __init__(self, input_dim: int = 8, hidden_dim: int = 32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

def featurize(smiles: str) -> list[float]:
    length = len(smiles)
    carbon_count = smiles.count("C")
    fluorine_count = smiles.count("F")
    oxygen_count = smiles.count("O")
    nitrogen_count = smiles.count("N")
    double_bonds = smiles.count("=")
    ring_marks = sum(ch.isdigit() for ch in smiles)
    fluorine_ratio = fluorine_count / max(length, 1)
    return [
        float(length),
        float(carbon_count),
        float(fluorine_count),
        float(oxygen_count),
        float(nitrogen_count),
        float(double_bonds),
        float(ring_marks),
        float(fluorine_ratio),
    ]

def _load_torch_file(path: Path):
    try:
        return torch.load(path, map_location="cpu", weights_only=True)
    except TypeError:
        return torch.load(path, map_location="cpu")

def _model_path(context: dict) -> Path:
    resources = context.get("resource_assets") or {}
    resource = resources.get("torch_checkpoint") or {}
    if resource.get("path"):
        return Path(resource["path"]) / "model.pt"

```

```

return Path(context["package_path"]) / "model" / "model.pt"

def load(context: dict):
    checkpoint_path = _model_path(context)
    if not checkpoint_path.is_file():
        raise FileNotFoundError(f"model checkpoint not found: {checkpoint_path}")

    checkpoint = _load_torch_file(checkpoint_path)
    metadata = {}
    state_dict = checkpoint
    if isinstance(checkpoint, dict) and "state_dict" in checkpoint:
        state_dict = checkpoint["state_dict"]
        metadata = checkpoint.get("metadata") or {}

    model = TorchRegressor(input_dim=int(metadata.get("input_dim", 8)))
    model.load_state_dict(state_dict)
    model.eval()
    return {"model": model, "metadata": metadata}

def predict(inputs: dict, context: dict, model=None) -> dict:
    if model is None:
        model = load(context)

    smiles = str(inputs.get("smiles", "")).strip()
    if not smiles:
        raise ValueError("smiles is required")

    features = featurize(smiles)
    tensor = torch.tensor([features], dtype=torch.float32)
    with torch.no_grad():
        value = float(model["model"](tensor).item())

    return {
        "prediction": {
            "property": "glass_transition_temperature",
            "value": round(value, 4),
            "unit": "degC",
            "model_version": context.get("version"),
        },
        "feature_summary": {
            "smiles": smiles,
            "feature_dim": len(features),
        },
    }

```

关键规则：

1. `load(context)` 只做模型和资源加载，返回可复用对象。
2. `predict(inputs, context, model)` 必须返回 dict。
3. 输出必须包含契约 `output_schema.required` 声明的字段。
4. 不要在 `predict` 中下载权重、读取用户目录或依赖外部密钥。
5. 错误直接抛出 `ValueError` / `FileNotFoundError`，平台会捕获并显示运行失败信息。

6. 小模型随 ZIP 打包的契约示例

适用于 `model/model.pt` 能放进 20MB ZIP 的情况。

```
contract_version: "0.1"
algorithm_id: polymer_torch_tg
name: Polymer Torch Tg Predictor
version: 0.1.0
algorithm_family: vertical_prediction
type: predictor
material_scope:
  - universal
task_scope:
  - COMPUTE_PREDICT
trigger_modes:
  - human_workflow
  - autoresearch
entrypoint: src.handler:predict
loader: src.handler:load
runtime:
  python: "3.11"
  resources:
    cpu: 1
    memory: 1Gi
    gpu: false
  timeout_seconds: 30
input_schema:
  fields:
    smiles: string
  required:
    - smiles
output_schema:
  fields:
    prediction: object
    feature_summary: object
  required:
    - prediction
sample_input_path: tests/sample_input.json
description: Torch 小模型示例，用于高分子 Tg 垂类预测。
developer: 模型开发者姓名或团队
developer_organization: 开发机构
developer_contact: contact@example.com
source_url: https://example.com/model-card
citation: "请填写论文、模型卡或内部报告引用。"
method_attributions:
  - name: PyTorch
    role: dependency
    organization: PyTorch Foundation
    description: 用于模型训练与推理的深度学习框架。
    url: https://pytorch.org/
visibility: private
```

`tests/sample_input.json` 示例：

```
{  
  "smiles": "C=C(F)F"  
}
```

requirements.txt 示例：

```
torch  
numpy
```

注意：当前本地 sandbox 构建策略是 `preinstalled_environment`。`requirements.txt` 会被校验并参与环境摘要，但不会自动帮你安装依赖。运维人员需要提前在后端 Python 运行环境中安装 `torch`。

7. 大权重受管资源契约示例

适用于权重大、文件后缀不是 ZIP 白名单、或多个资源文件需要统一管理的情况。

```
contract_version: "0.2"
algorithm_id: polymer_torch_tg
name: Polymer Torch Tg Predictor
version: 0.2.0
algorithm_family: vertical_prediction
type: predictor
material_scope:
  - universal
task_scope:
  - COMPUTE_PREDICT
trigger_modes:
  - human_workflow
  - autoresearch
entrypoint: src.handler:predict
loader: src.handler:load
runtime:
  python: "3.11"
  resources:
    cpu: 1
    memory: 1Gi
    gpu: false
  timeout_seconds: 60
input_schema:
  fields:
    smiles: string
  required:
    - smiles
output_schema:
  fields:
    prediction: object
    feature_summary: object
  required:
    - prediction
resource_assets:
  - key: torch_checkpoint
    label: Torch 模型权重
    required: true
    asset_role: resource
    data_kind: binary
    parser: binary.v1
    resource_type: torch_checkpoint
    required_files:
      - model.pt
      - feature_stats.json
    binding_required: true
    description: Torch 权重与推理所需归一化参数目录。
sample_input_path: tests/sample_input.json
description: 使用平台受管资源加载 Torch 权重的垂类预测模型。
developer: 模型开发者姓名或团队
developer_organization: 开发机构
developer_contact: contact@example.com
visibility: private
```

后端资源目录示例：


```
.runtime/algorithm-resources/polymer_torch_tg/  
model.pt  
feature_stats.json
```

资源路径必须满足以下条件：

1. 路径在 `.runtime/algorithm-resources` 下，或在环境变量 `POLYAGENT_ALGORITHM_RESOURCE_ROOTS` 允许的根目录下。
2. `required_files` 使用相对路径，不能是绝对路径，不能包含 `..`。
3. 登记资源时平台会检查目录和必需文件是否存在。
4. 算法运行时通过 `context["resource_assets"]["torch_checkpoint"]["path"]` 获取目录。

8. 网页接入流程

推荐首次接入使用网页打包助手。

1. 打开 任务提交 -> 垂类预测模型。
2. 进入 上传部署 Tab。
3. 选择 Python 脚本自动打包。
4. 填写基础信息：
 - 算法 ID：例如 `polymer_torch_tg`，同一算法不同版本保持一致。
 - 名称：例如 `Polymer Torch Tg Predictor`。
 - 版本：例如 `0.1.0`，升级时递增。
 - 类型：通常选择 `predictor`。
 - 材料范围：不确定时选择 `universal`。
 - 触发方式：需要人工调用和 `AutoResearch` 都可用时选择 `human_workflow` 和 `autoresearch`。
5. 填写入口函数：
 - 入口函数：`src.handler:predict`
 - 加载函数：`src.handler:load`
6. 填写输入契约：
 - 字段名：`smiles`
 - 类型：`string`
 - 必填：是
7. 填写输出契约：
 - 字段名：`prediction`
 - 类型：`object`
 - 必填：是
 - 可增加 `feature_summary`、`uncertainty`、`model_info` 等字段。

8. 填写开发者和来源信息：

- developer
- developer_organization
- developer_contact
- source_url
- citation

9. 上传源码文件：

- 至少上传一个 .py 文件。
- 单文件 handler.py 会被平台放入 src/handler.py。

10. 上传 requirements.txt。

11. 填写样例输入 JSON，例如：

```
{  
  "smiles": "C=C(F)F"  
}
```

1. 检查页面生成的 polyagent.algorithm.yaml 预览。

2. 点击 校验、部署并激活。

3. 等待流程完成：

- 生成或上传标准 ZIP
- 校验契约和样例 dry-run
- 构建运行摘要
- 部署 staging
- 激活版本

1. 完成后在 测试调用 Tab 选择算法和版本，输入样例并运行。

如果模型有大权重，先执行“资源管理”流程，再上传算法包。

9. 资源管理流程

大模型、tokenizer、数据库、归一化参数等建议使用资源管理。

1. 运维人员把资源文件放到后端允许路径，例如：

```
mkdir -p .runtime/algorithm-resources/polymer_torch_tg  
cp model.pt feature_stats.json .runtime/algorithm-resources/polymer_torch_tg/
```

1. 打开 垂类预测模型 -> 资源管理。

2. 填写：

- 算法 ID：polymer_torch_tg
- 资源 Key：torch_checkpoint

- 名称：Polymer Torch checkpoint
- 资源类型：torch_checkpoint
- 服务器路径：资源目录绝对路径
- 必需文件：

```
model.pt  
feature_stats.json
```

1. 点击 登记资源。
2. 在资源表点击 检查，确认状态为 active。
3. 在算法契约 resource_assets 中使用同一个 key，即 torch_checkpoint。
4. 校验算法包时，平台会自动按 algorithm_id + asset_key 查找 active 资源；如果页面或 API 提供绑定参数，也可以显式绑定 resource_id。

10. CLI 打包流程

适合已有代码目录或需要本地自动化交付的团队。

目录示例：

```
my_torch_model/  
  README.md  
  requirements.txt  
  src/  
    handler.py  
  model/  
    model.pt  
  tests/  
    sample_input.json
```

打包命令：

```
python scripts/pack_algorithm.py \  
  --algorithm-id polymer_torch_tg \  
  --name "Polymer Torch Tg Predictor" \  
  --version 0.1.0 \  
  --entrypoint src.handler:predict \  
  --loader src.handler:load \  
  --source-dir my_torch_model \  
  --requirements my_torch_model/requirements.txt \  
  --sample-input my_torch_model/tests/sample_input.json \  
  --input-schema '{"fields":{"smiles":"string"},"required":["smiles"]}' \  
  --output-schema '{"fields":{"prediction":"object","feature_summary":"object"},"required":["prediction"]}' \  
  --output /tmp/polymer_torch_tg-0.1.0.zip
```

大权重受管资源版本可以增加：

```
python scripts/pack_algorithm.py \
  --algorithm-id polymer_torch_tg \
  --name "Polymer Torch Tg Predictor" \
  --version 0.2.0 \
  --entrypoint src.handler:predict \
  --loader src.handler:load \
  --source-dir my_torch_model \
  --requirements my_torch_model/requirements.txt \
  --sample-input my_torch_model/tests/sample_input.json \
  --input-schema '{"fields":{"smiles":"string"},"required":["smiles"]}' \
  --output-schema '{"fields":{"prediction":"object","feature_summary":"object"},"required":["prediction"]}' \
  --resource-assets '[{"key":"torch_checkpoint","label":"Torch 模型权重","required":true,"asset_role":"resource","data_kind":"binary","parser":"binary.v1","resource_type":"torch_checkpoint","required_files":["model.pt","feature_stats.json"],"binding_required":true}]' \
  --output /tmp/polymer_torch_tg-0.2.0.zip
```

然后在网页 上传部署 中选择 标准 ZIP 直接上传，上传生成的 ZIP。

11. API 接入流程

网页流程背后对应这些接口，适合自动化平台集成。

操作	方法与路径
下载模板 ZIP	GET /api/v1/research-engine/algorithm-packages/template
网页打包助手提交	POST /api/v1/research-engine/algorithm-packages:pack
标准 ZIP 上传	POST /api/v1/research-engine/algorithm-packages
校验包	POST /api/v1/research-engine/algorithm-packages/{package_id}:validate
构建包	POST /api/v1/research-engine/algorithm-packages/{package_id}:build
部署版本	POST /api/v1/research-engine/algorithms/{algorithm_id}/versions/{version_id}:deploy
激活版本	POST /api/v1/research-engine/algorithms/{algorithm_id}/versions/{version_id}:activate
查询版本	GET /api/v1/research-engine/algorithms/{algorithm_id}/versions
测试调用 JSON 输入	POST /api/v1/research-engine/algorithm-runs
测试调用文件输入	POST /api/v1/research-engine/algorithm-runs:multipart
查询运行记录	GET /api/v1/research-engine/algorithm-runs
查询运行产物	GET /api/v1/research-engine/algorithm-runs/{run_id}/artifacts
登记资源	POST /api/v1/research-engine/algorithm-resources
检查资源	POST /api/v1/research-engine/algorithm-resources/{resource_id}:check

标准 ZIP 上传后的典型顺序：

```
upload -> validate -> build -> deploy -> activate -> run
```

12. 使用者调用流程

模型激活后，普通使用者按以下步骤调用：

1. 打开 任务提交 -> 垂类预测模型。
2. 进入 测试调用 Tab。
3. 选择算法，例如 `polymer_torch_tg`。
4. 选择版本；默认使用 active 版本。
5. 按页面字段填写输入，例如 `smiles = C=C(F)F`。
6. 如算法声明了文件输入，按控件上传对应文件。
7. 点击运行。
8. 查看：
 - `prediction`：主要预测结果。
 - `feature_summary`：特征摘要或模型解释信息。
 - `artifact`：文件产物下载入口。
 - `runtime`：耗时、状态、digest 等追溯信息。
9. 在 运行记录 Tab 可按算法、版本、状态和日期检索历史调用。

13. 文件输入与文件产物

如果 Torch 模型需要表格、谱图、图像或二进制输入，可以使用 `contract_version: "0.2"` 并声明 `input_assets`。

表格输入示例：

```
input_assets:
- key: feature_table
  label: 特征表
  required: true
  asset_role: input
  data_kind: table
  parser: table.v1
  extensions:
    - .csv
    - .xlsx
  max_size_bytes: 10485760
  sample_path: tests/sample_assets/feature_table.csv
```

运行时读取方式：

```
def predict(inputs: dict, context: dict, model=None) -> dict:
    input_files = context.get("input_files") or {}
    parsed_inputs = context.get("parsed_inputs") or {}

    raw_table_path = input_files.get("feature_table")
    parsed_table = parsed_inputs.get("feature_table")
    ...
```

如果模型会输出文件，使用 `result_envelope: polyagent_run_result.v1`，把文件写入 `context["output_dir"]` 并返回 artifact 列表。

```
def predict(inputs: dict, context: dict, model=None) -> dict:
    output_dir = Path(context["output_dir"])
    report_path = output_dir / "prediction_report.json"
    report_path.write_text(json.dumps({"ok": True}, ensure_ascii=False), encoding="utf-8")

    return {
        "output_summary": {
            "prediction": {"value": 101.2, "unit": "degC"}
        },
        "artifacts": [
            {
                "key": "prediction_report",
                "path": "prediction_report.json",
                "name": "prediction_report.json",
                "mime_type": "application/json",
                "artifact_type": "result_json"
            }
        ]
    }
```

对应契约：

```
result_envelope: polyagent_run_result.v1
output_assets:
  - key: prediction_report
    label: 预测报告
    required: false
    asset_role: output
    data_kind: json
    parser: json.v1
    artifact_type: result_json
    mime_type: application/json
    extensions:
      - .json
```

输出文件限制：

限制	当前值
单个输出 artifact	50MB

限制	当前值
单次运行输出 artifact 总量	200MB

14. 版本治理

状态	含义	使用建议
uploaded	ZIP 已上传	等待校验
validated	契约和样例 dry-run 通过	可以构建
built	构建摘要已生成	可以部署
deployed_staging	runtime 已登记	可以激活或测试治理
active	当前可被选择和调用的版本	生产/团队使用版本
frozen	保留版本但禁止新任务选择	有疑问但需保留追溯
decommissioned	下线版本	不再用于新任务

升级模型时不要覆盖旧包。使用同一个 `algorithm_id` 和新的 `version` 上传。激活新版本后，旧 `active` 版本会回到 `staging`，可按需回滚、冻结或下线。

15. 自测脚本

上传前建议在本地目录运行一次最小自测。

```
from src.handler import load, predict

context = {
    "package_path": ".",
    "resource_assets": {},
    "input_files": {},
    "parsed_inputs": {},
    "output_dir": "/tmp/polyagent_algorithm_test",
    "version": "0.1.0",
}

model = load(context)
result = predict({"smiles": "C=C(F)F"}, context, model)
assert isinstance(result, dict)
assert "prediction" in result
print(result)
```

运行：

```
cd my_torch_model
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
python smoke_test.py
```

如果本地自测依赖 GPU 才能通过，不建议直接接入当前默认垂类预测运行时；应先改为 CPU 推理或由平台侧扩展 GPU runtime。

16. 常见错误与处理

错误现象	常见原因	处理方式
P0 仅支持 Python 3.11	契约 runtime 写了其他版本	改为 <code>runtime.python: "3.11"</code>
不支持的文件类型: model/ model.pth	ZIP 后缀白名单未包含 .pth	改成 .pt，或放入受管资源
算法包超过 20MB 限制	权重随 ZIP 打包过大	使用资源管理登记 mounted path
No module named torch	后端环境未安装 Torch	运维在后端 Python 环境安装 torch
入口函数必须使用 module:function 格式	entrypoint 或 loader 写法错误	使用 <code>src.handler:predict / src.handler:load</code>
predict() 必须返回 dict	返回了 list、float、字符串或 numpy 类型	包装成标准 dict，并把 numpy scalar 转成 Python 类型
输出缺少必填字段	返回结果缺少 <code>output_schema.required</code> 字段	修改 <code>predict</code> 输出或调整契约
<code>resource asset ...</code> 缺少平台 资源绑定	<code>resource_assets</code> required 但未登记 active 资源	在资源管理登记并检查资源
<code>required_files</code> 包含非法路径	必需文件写了绝对路径或 ...	改为资源目录内相对路径
<code>algorithm execution timed out</code>	模型加载或推理超过超时	增加 <code>timeout_seconds</code> ，优化加载，或减 小样例输入
<code>sandbox runtime did not return JSON</code>	程序提前崩溃或 stdout 干扰过多	查看版本日志和 <code>traceback tail</code>

17. 接入验收标准

一个 Torch 垂类模型达到可交付状态，应满足：

1. 标准 ZIP 可以上传并校验通过。
2. 样例 dry-run 能返回 `prediction`。
3. 依赖在后端环境中已验证可 `import`。
4. 如果使用受管资源，资源状态为 `active`。

5. 版本完成部署并激活。
6. 测试调用 中至少完成一次真实输入运行。
7. 运行记录 中能看到输入、输出、状态、耗时和版本信息。
8. 模型来源、开发者、机构和引用信息已填写。
9. README 写明适用范围、输入含义、输出单位、已知限制和失败处理。

18. 推荐交付模板

给平台接入人员的交付包说明建议包含：

```
算法 ID: polymer_torch_tg
算法名称: Polymer Torch Tg Predictor
版本: 0.1.0
模型类型: PyTorch state_dict
运行硬件: CPU
Python 版本: 3.11
主要依赖: torch, numpy
输入字段: smiles: string, required
输出字段: prediction: object, feature_summary: object
权重位置: 随 ZIP 的 model/model.pt, 或受管资源 torch_checkpoint/model.pt
样例输入: tests/sample_input.json
开发者: ...
开发机构: ...
联系方式: ...
来源链接: ...
推荐引用: ...
适用范围: ...
不适用范围: ...
```

19. 接入建议

优先走最小可用路径：先用一个 CPU state_dict 小模型和单字段 JSON 输入跑通

upload -> validate -> build -> deploy -> activate -> run。确认链路稳定后，再增加文件输入、输出 artifact、大权重资源、版本治理和 AutoResearch 调用。

如果模型包里包含第三方框架、外部论文方法、机构模型或开源依赖，应填写 developer、developer_organization、source_url、citation 和 method_attributions，让模型中心、详情页和预测结果页面能显示清晰来源。